the Redis calls (which Sidekiq advises you limit to 1,000):

```
order_ids = Order.where(fulfillment_request_id: nil).pluck(:id)
array_of_args = order_ids.zip # Turns each element into an array
RequestOrderFulfillmentJob.perform_bulk(order_ids, batch_size: 500)
```

You could put this code into a nightly job or scheduled task.

Scheduling Jobs Is Available with the Enterprise Version

To schedule jobs to run at a given time—such as a job containing the code above—you can use a third-party gem like Sidekiq Scheduler,[28] or you can use the Periodic Jobs feature of Sidekiq Enterprise,[29] which, as the name implies, requires a paid Sidekiq Enterprise plan. Sometimes the best way to solve a problem is to pay someone else to solve it.

The advantage of Periodic Jobs is that it's part of Sidekiq and is supported directly. It's reliable and will always work. The downside is that it requires an Enterprise plan which you may not be able to afford. While Sidekiq Scheduler is free and does work pretty well, it tends to lag behind Sidekiq in terms of features and support. This could prevent you from upgrading Sidekiq in the future.

Sidekiq Enterprise and Pro Plans Solve Real Problems

While I'm neither affiliated with nor compensated by Sidekiq's author, it is worth knowing that the paid plans (Pro and Enterprise) for Sidekiq aren't just there to support the author's work. They fund real features that solve real problems. Periodic Jobs is one of them, but another is reliability when fetching jobs to process.

We discussed earlier that it's possible for jobs to be lost due to the way Redis works. It's also possible to lose a job if Sidekiq has fetched a job to process, but then dies before completing the job *and then* is unable to push the job back to Redis to try again later. While this may be unlikely, it's